

UNIT 3: Inheritance, Interfaces and Packages

Q1. Compare 'this' keyword Vs 'Super' keyword. Explain super keyword with suitable Example. [07 Marks]

(Winter-2024)

Answer:

Comparison: this vs super

Feature	this keyword	super keyword
Purpose	Current class ની object ને refer કરે છે.	Immediate parent (super) class ની members ને refer કરે છે.
Variables	Current class ની instance variable ને refer કરે છે.	Parent class ની instance variable ને access કરવા વપરાય છે.
Methods	Current class ની method invoke કરવા વપરાય છે.	Parent class ની overridden method call કરવા વપરાય છે.
Constructor Call	Current class ની constructor call માટે: this()	Parent class ની constructor call માટે: super()

Explanation of super keyword with Example:

- Subclass માં overridden method કે hidden variable હોવા છતાં parent class ની members access કરવા super વપરાય છે.
- Parent class ની constructor ને call કરવા constructor ની પ્રથમ લાઇનમાં super() લખાય છે.

Java

```
class A {  
    String name = "Class A";  
    A() {  
        System.out.println("Super class default constructor called..");  
    }  
}
```

```

    }

    public void display() {
        System.out.println("Class A display method called..");
    }
}

class B extends A {
    String name = "Class B";

    B() {
        super(); // Call superclass constructor

        System.out.println("Sub class default constructor called..");
    }

    void printName() {
        System.out.println("Name from subclass: " + name);

        System.out.println("Name from Superclass: " + super.name); // Superclass variable
        super.display(); // Superclass method
    }
}

```

```

public class SuperDemo {
    public static void main(String args[]) {
        B b1 = new B();

        b1.printName();
    }
}

```

Output:

Plaintext

Super class default constructor called..

Sub class default constructor called..

Name from subclass: Class B

Name from Superclass: Class A

Class A display method called..

Q2. Define Inheritance. List out types of it. Explain multilevel and hierarchical inheritance with suitable example. [07 Marks]

(Summer-2024, Winter-2025)

Answer:

Definition: Inheritance એ એક એવી પ્રક્રિયા છે જેના દ્વારા derived class બીજા existing base class ની properties અને methods મેળવી શકે છે. Java માં આ માટે extends keyword વપરાય છે.

Types of Inheritance:

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance (ઇન્ટરફેસ દ્વારા સપોર્ટેડ)
5. Hybrid Inheritance

1. Multilevel Inheritance: જ્યારે એક class બીજા derived class માંથી derive થાય છે (Chain of inheritance).

Plaintext

[Class A] -> [Class B] -> [Class C]

2. Hierarchical Inheritance: જ્યારે એક જ super class માંથી એક કરતાં વધુ sub classes derive થાય છે.

Plaintext

[Class A]

/ \

[Class B] [Class D]

Example Program:

Java

```
class A {  
    public void displayA() {  
        System.out.println("class A method");  
    }  
}
```

// Single / Base for multilevel

```
class B extends A {  
    public void displayB() {  
        System.out.println("class B method");  
    }  
}
```

// Multilevel Inheritance: C extends B which extends A

```
class C extends B {  
    public void displayC() {  
        System.out.println("class C method");  
    }  
}
```

// Hierarchical Inheritance: D also extends A directly

```
class D extends A {  
    public void displayD() {  
        System.out.println("class D method");  
    }  
}
```

```
public class InheritanceDemo {
```

```

public static void main(String[] args) {

    C c = new C();

    c.displayA(); // Inherited from A

    c.displayB(); // Inherited from B

    c.displayC();


    D d = new D();

    d.displayA(); // Inherited from A

    d.displayD();

}
}

```

Output:

```

Plaintext

class A method

class B method

class C method

class A method

class D method

```

Q3. Define Interface in Java. Write a Java program to demonstrate Multiple inheritance using interfaces in Java. [07 Marks]

(Summer-2026)

Answer:

Definition: Interface એ abstract methods અને static constants નું collection છે. તે fully abstraction આપે છે. By default fields public static final અને methods public abstract હોય છે. Class interface ને implement કરવા implements keyword વાપરે છે.

Multiple Inheritance: Java classes multiple inheritance સપોર્ટ કરતું નથી, પરંતુ class એક કરતાં વધુ interfaces implement કરીને multiple inheritance પ્રાપ્ત કરી શકે છે.

Java Program:

Java

```
interface A {  
    void getdata(String name);  
}  
  
interface B {  
    void setdata();  
}  
  
class InheritInterface implements A, B {  
    String display;  
  
    public void getdata(String name) {  
        display = name;  
    }  
  
    public void setdata() {  
        System.out.println(display);  
    }  
  
    public static void main(String args[]) {  
        InheritInterface obj = new InheritInterface();  
        obj.getdata("Welcome TO Heaven");  
        obj.setdata();  
    }  
}
```

Output:

Plaintext

Welcome TO Heaven

Q4. Define abstract class in JAVA with example. [04 Marks]

(Summer-2025)

Answer:

- **Definition:** જે class ને abstract keyword સાથે declare કરવામાં આવે તેને **Abstract Class** કહે છે.
- **Rules:**
 1. તેનો object બનાવી શકાતો નથી (cannot be instantiated).
 2. તેમાં abstract methods અને concrete methods (methods with body) બંને હોઈ શકે છે.
 3. તેને inherit કરનાર class એ તમામ abstract methods implement કરવી પડે છે.

Example:

Java

```
abstract class A {  
    abstract void abs_method(); // Abstract method  
  
    public void display() { // Concrete method  
        System.out.println("This is concrete method.");  
    }  
}  
  
class DemoAbstract extends A {  
    void abs_method() {  
        System.out.println("This is an abstract method.");  
    }  
  
    public static void main(String[] args) {  
        DemoAbstract abs = new DemoAbstract();  
        abs.abs_method();  
    }  
}
```

```
abs.display();  
}  
}
```

Output:

Plaintext

This is an abstract method..

This is concrete method..

Q5. Define an interface and mention how it differs from a class. [03 Marks]

(Winter-2025)

Answer:

Definition: Interface એ abstract methods નું collection છે જે 100% abstraction આપે છે.

Class	Interface
Class નો object instantiate કરી શકાય છે.	Interface નો object instantiate કરી શકાતો નથી.
Concrete methods (definition સાથે) હોય છે.	બધી methods by default abstract હોય છે.
Constructors હોઈ શકે છે.	કોઈ constructors હોતા નથી.
Variables instance variables હોય છે.	Variables by default public static final હોય છે.

Q6. Define package and write the syntax to create a package with example. [04 Marks]

(Summer-2025)

Answer:

Definition: Package એ related classes, interfaces અને sub-packages નું એક જૂથ (group) છે. તે naming conflict રોકે છે અને access protection આપે છે.

Syntax to create package:

Java

```
package packagename;
```

(Package statement file માં સૌથી પહેલી non-commented લાઇન હોવી જોઈએ).

Example:

Java

```
package mypack;
```

```
public class TestClass {  
    public void show() {  
        System.out.println("Package created successfully!");  
    }  
}
```

- **Compile command:** javac -d . TestClass.java

Q7. Define package in JAVA. Write a Java program to create package. [03 Marks]

(Summer-2026)

Answer:

Definition: Package એ સમાન પ્રકારના classes અને interfaces નું container છે.

Program:

Java

```
package mypack;
```

```
public class DemoPackage {  
    public static void main(String[] args) {  
        System.out.println("Welcome to mypack Package");  
    }  
}
```

- **Execution:** javac -d . DemoPackage.java પછી java mypack.DemoPackage.

Output:

Plaintext

Welcome to mypack Package

Q8. Discuss in detail the Object class methods (toString(), equals(), hashCode()) and show their overriding in a Java class Student. [07 Marks]

(Winter-2025)

Answer:

Java માં java.lang.Object એ તમામ classes નો default super class છે.

1. **toString():** Object નું string representation રિટર્ન કરે છે.
2. **equals(Object obj):** બે objects ની equality (content value) compare કરે છે.
3. **hashCode():** Object નું unique integer memory-hash code આપે છે.

Overriding in Student Class:

Java

```
class Student {
```

```
    int rollNo;
```

```
    String name;
```

```
    Student(int rollNo, String name) {
```

```
        this.rollNo = rollNo;
```

```
        this.name = name;
```

```
    }
```

```
    // Overriding toString()
```

```
    public String toString() {
```

```
        return "Student[RollNo=" + rollNo + ", Name=" + name + "];"
```

```
    }
```

```
    // Overriding equals()
```

```
    public boolean equals(Object obj) {
```

```

        Student s = (Student) obj;

        return this.rollNo == s.rollNo && this.name.equals(s.name);
    }

    // Overriding hashCode()
    public int hashCode() {
        return rollNo;
    }
}

public class ObjectMethodsDemo {
    public static void main(String args[]) {
        Student s1 = new Student(101, "Sujal");
        Student s2 = new Student(101, "Sujal");

        System.out.println(s1); // Internally calls toString()
        System.out.println("s1 equals s2: " + s1.equals(s2));
        System.out.println("s1 Hashcode: " + s1.hashCode());
    }
}

```

Output:

Plaintext

Student[RollNo=101, Name=Sujal]

s1 equals s2: true

s1 Hashcode: 101

Q9. Explain Abstract class in Java with suitable example. [04 Marks]

(Summer-2026)

Answer:

- જે class માં implementation વગરની methods (abstract methods) હોય તેને **Abstract Class** તરીકે define કરવામાં આવે છે.
- Abstract class નો reference variable બનાવી શકાય છે પરંતુ તેનો object create કરી શકાતો નથી.

Example:

Java

```
abstract class Bike {

    abstract void run(); // Abstract method


    void changeGear() { // Concrete method

        System.out.println("Gear changed.");

    }

}

class Honda extends Bike {

    void run() {

        System.out.println("Honda bike is running safely.");

    }

}

public class TestAbstract {

    public static void main(String args[]) {

        Bike obj = new Honda();

        obj.run();

        obj.changeGear();

    }

}
```

Output:

Plaintext

Honda bike is running safely.

Gear changed.

Q10. Explain Final class in Java with suitable example. [04 Marks]

(Summer-2026)

Answer:

- **Concept:** જ્યારે કોઈ class ની આગળ final keyword લખવામાં આવે, ત્યારે તે class ને **Inherit (Extend) કરી શકાતો નથી.**
- સુરક્ષાના કારણોસર class ની functionality અટકાવવા માટે final વપરાય છે.

Example:

Java

```
final class SecureBase {  
    void display() {  
        System.out.println("Inside Final Class");  
    }  
}
```

// The following inheritance will cause Compile Time Error:

```
// class SubClass extends SecureBase { }
```

```
public class FinalClassDemo {  
    public static void main(String args[]) {  
        SecureBase obj = new SecureBase();  
        obj.display();  
    }  
}
```

Output:

Plaintext

Inside Final Class

Q11. Explain Inheritance and its types in Java with suitable diagrams. [07 Marks]

(Summer-2026)

Answer:

Definition: Inheritance એ parent class ની features re-use કરવાની પદ્ધતિ છે.

Types of Inheritance:

1. **Single:** One parent, one child.

Plaintext

[Class A]

|

[Class B]

2. **Multilevel:** Chain of inheritance.

Plaintext

[Class A] -> [Class B] -> [Class C]

3. **Hierarchical:** Multiple children from single parent.

Plaintext

[Class A]

/ | \

[B] [C] [D]

4. **Multiple (via Interfaces):** One child from multiple parents.

Plaintext

[Interface A] [Interface B]

\ /

[Class C]

Q12. Explain abstract class with suitable example. [03 Marks]

(Summer-2024)

Answer:

Java

```

abstract class Animal {
    abstract void sound();
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class MainApp {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}

```

Output:

Plaintext

Dog barks

Q13. Explain method overriding with Java example. [04 Marks]

(Winter-2025)

Answer:

- **Method Overriding:** જ્યારે subclass માં એવી method define કરવામાં આવે જેનું નામ, arguments list અને return type superclass જેવો જ હોય ત્યારે subclass method superclass ની method ને override કરે છે.
- તે **Runtime Polymorphism** માટે વપરાય છે.

Example:

Java

```
class A {  
    public void display() {  
        System.out.println("Class A");  
    }  
}  
  
class B extends A {  
    public void display() { // Overriding display()  
        System.out.println("Class B");  
    }  
}  
  
public class Trial {  
    public static void main(String args[]) {  
        A a = new A();  
        A b = new B(); // Runtime resolution  
        a.display();  
        b.display();  
    }  
}
```

Output:

Plaintext

Class A

Class B

**Q14. Explain package and list out steps to create package with suitable example.
[07 Marks]**

(Winter-2024)

Answer:

Explanation of Package: Package અને related classes અને interfaces નું collection છે.

- **Built-in Packages:** java.lang, java.util, java.io
- **User-defined Packages:** Programmers દ્વારા બનાવવામાં આવેલ custom packages.

Steps to Create Package:

1. Java source file ની શરૂઆતમાં package <packagename>; લખો.
2. Class અને તેના members ને public declare કરો.
3. Compilation command: javac -d . Filename.java
4. બીજા program માં access કરવા માટે: import packagename.*; અથવા import packagename.ClassName;

Example Program:

Java

```
// File: DemoPackage.java
```

```
package mypack;
```

```
class Book {
```

```
String bookname = "Complete Reference";
```

```
String author = "Herbert";
```

```
void show() {
```

```
System.out.println("Book name is :: " + bookname + "\nand author name is :: " +
author);
```

}

}

```
public class DemoPackage {
```

```
public static void main(String[] args) {
```

```
Book b1 = new Book();
```

```
b1.show();
```

```
}  
}
```

Output:

Plaintext

Book name is :: Complete Reference

and author name is :: Herbert

Q15. Explain super keyword with example. [04 Marks]

(Winter-2025)

Answer:

super keyword નો ઉપયોગ subclass માંથી immediate superclass members (variables, methods, constructors) ને refer કરવા માટે થાય છે.

Example:

Java

```
class SuperClass {  
    int num = 100;  
}
```

```
class SubClass extends SuperClass {  
    int num = 200;
```

```
    void printNumbers() {  
        System.out.println("Subclass num: " + num);  
        System.out.println("Superclass num: " + super.num); // super keyword  
    }  
}
```

```
public class TestSuper {  
    public static void main(String args[]) {
```

```
SubClass obj = new SubClass();  
obj.printNumbers();  
}  
}
```

Output:

Plaintext

Subclass num: 200

Superclass num: 100

Q16. Explain use of packages in java with java programme. [04 Marks]

(Winter-2025)

Answer:

Uses of Packages:

1. Code reusability વધારવા માટે.
2. Naming conflicts દૂર કરવા (દા.ત., બે અલગ package માં same class name હોઈ શકે).
3. Class members ને access protection પૂરું પાડવા.

Program:

Java

```
package pack;
```

```
public class A {  
    public void display() {  
        System.out.println("Welcome to package pack...");  
    }  
}
```

```
public static void main(String args[]) {  
    A obj = new A();  
    obj.display();  
}
```

```
}
```

Output:

Plaintext

Welcome to package pack...